

低代码可视化平台架构设计内丹

1. 课程代码



demo.zip
1.85KB



课后咨询老师领取

课程目标



- 了解无代码平台核心业务模型
- 掌握项目工程化设计思维，包括打包构建流程
- 掌握前端企业及项目架构设计思想
- 掌握技术选型与评估，并能独立设计开发满足业务的组件以及封装库

2. 低代码平台工程架构与设计

2.1 背景介绍

近年来，无代码平台与可视化相关平台日益火热。

无代码允许开发人员低成本通过拖拉拽的方式快速构建企业内部系统或落地页，常见的无代码平台能力包括：审批流、应用构建等。其在企业开发提效与运营方面取得了不错的成绩。例如国外的：

Coda、**Airtable**、**Notion** 等产品，例如国内的：**明道云**、**极简云**、**ClickPaaS** 等产品。构建无代码平台是一件有挑战性的事情，这节实战课会从业务与技术架构入手，带你彻底弄懂无代码平台研发。

可视化平台允许数据分析人员通过拖拉拽的方式快速组建仪表盘或数据分析大盘，其在企业运营方面发挥重要作用。例如国外的：**PowerBI**、**Monday**、**Coda** 等产品，国内的：**观远数据**、**BDP**、**FineBI** 等产品。构建可视化 SaaS 是一件非常有趣的事情，这节实战课会从业务与技术架构出发，带你从零完成可视化图表数据协议设计、图表渲染器开发、图表编排等功能。

当无代码遇上可视化，相信会有更多火花，没错，我们课程将这两大火热业务方向进行融合，带你真正体验大厂企业级项目的设计、开发与流程管控。

开发过程中，我们会严格按照大厂开发流程，从项目搭建、规范约定、技术选型与评估、需求分析到开发，每个流程细节我们都不会放过。技术选型方面，我们会选用 Vue3 作为技术栈，状态管理选用 Pinia，编辑器开发我们会基于 tiptap 来展开，Echarts 开发图表相关内容，除此之外，我们会向大家展示如何更优雅更松散处理复杂表单的场景。相信你学完后，一定会感慨自己就这样又更上了一层楼！



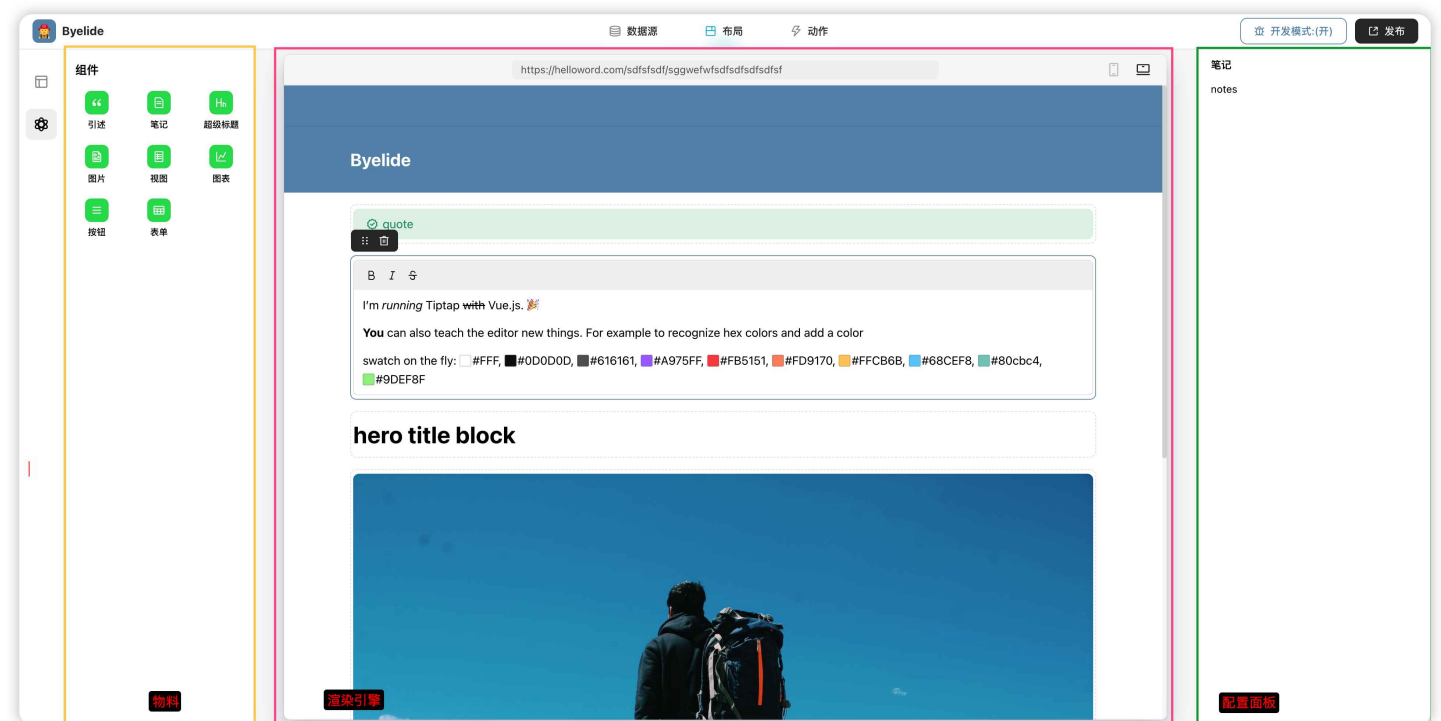
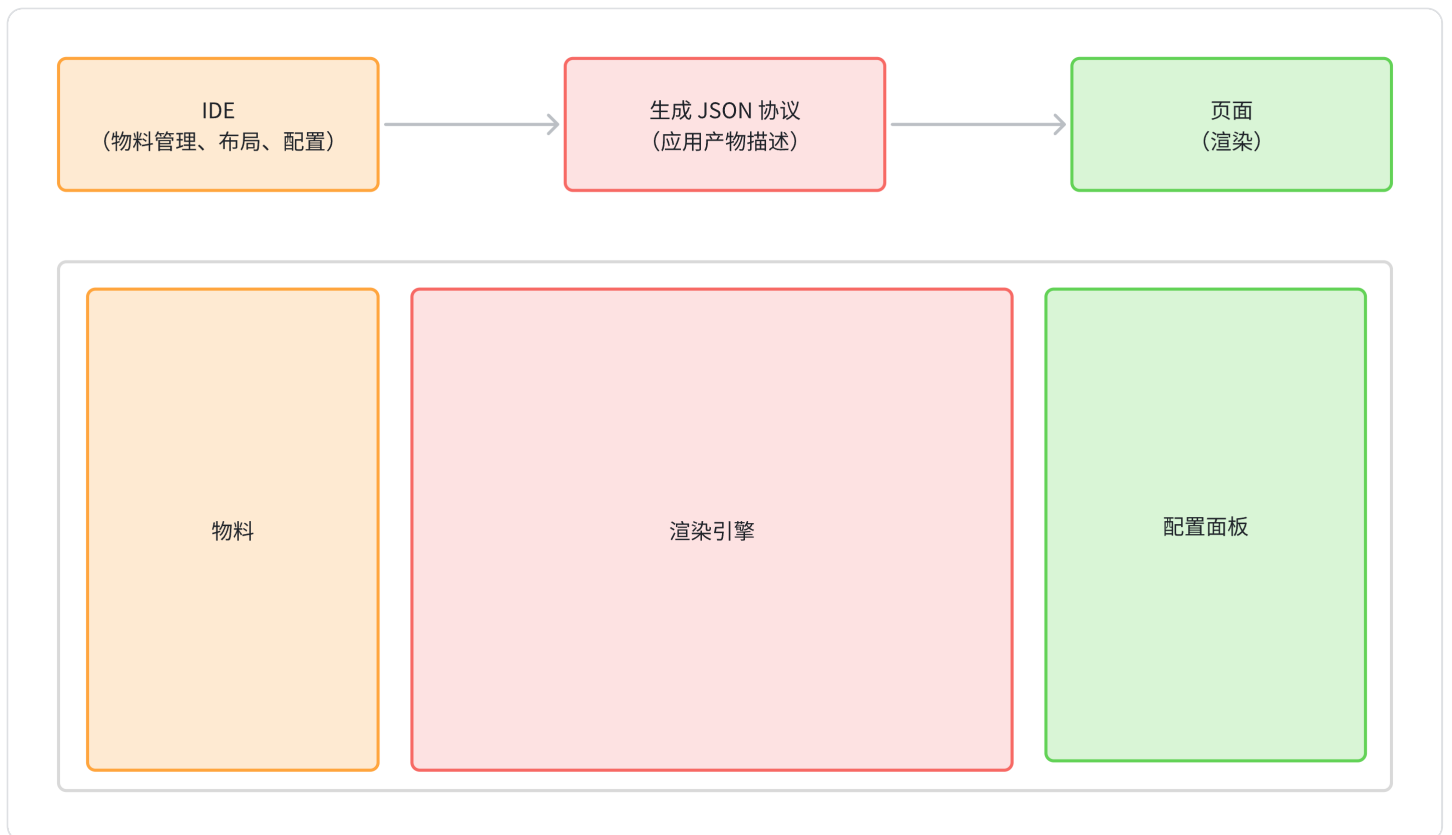
请注意无代码与低代码的主要区别：

1. 无代码一般面向**普通用户**，可以没有编程经验，可以以最简单形式搭建应用。而低代码需要用户具备一定的编程能力。
2. 在实现上，低代码比无代码更复杂，低代码还会存在例如 **DSL** 以及代码植入的能力，低代码的代码片段植入思路接近微前端。

2.2 核心能力

- 可视化配置面板
- 具有可拓展能力：组件、模板、逻辑复用
- 生命周期管理：开发管理、页面管理、部署管理

2.3 架构设计



2.3.1 物料堆

包含可供使用的物料，通常物料的组织与消费是插件化的，还有很多产品将物料组织发布到插件市场，可供用户自行拓展，这都是跟插件化思想与 **js 沙盒** 设计离不开的。

<https://juejin.cn/post/6981374562877308936>

2.3.2 编排

- 微内核思想：操作的是DSL（json树）， $f(state) \rightarrow view$

- 组合和渲染层隔离
- render runtime(渲染引擎sdk) + dsl(json) = 页面
- 事件：DND拖拽、Mouse Event
- 画布分层技术：借鉴浏览器渲染模型，使用冒泡机制，走到所有层
 - 第一层：div，负责渲染，render(dsl, document.querySelector("#root"))
 - 第二层：加一层div，只负责处理右键事件
 - 第三层：加一层div，只处理快捷键
 - event bus：所有层可通过 event bus 进行通信
- 画布功能及拓展：
 - 简单编排
 - 物料均为块级，操作简便，不涉及复杂布局问题
 - 画布灵活编排
 - 无限画布、引导线（衫格）、吸附对齐、旋转、快捷键、右键、缩放
 - 无限画布：监听滚动事件，每次给画布加宽带
 - 引导线：使用div画线（高度和宽带为1px）、绝对定位可拖动，下方
 - 吸附对齐：计算想尽的dom节点，定6个点和3个线，距离相近时，设置距离为0

2.3.3 配置面板

配置面板就是对物料进行配置，我们需要将数据、视图、操作解耦

- 功能：撤销、重做、预览、提测、发布
 - 重做（undo/redo）：使用队列，指针移动
 - 预览：render(dsl: {type, key, props}, animate {}, actions: {}, attrs: {}, children [])
- 中间层：权限控制、数据操作（转换），暴露部分api

这块比较考验大家对于复杂表单的设计实现

2.3.4 渲染器与出码

通常平台会提供 schema 渲染页面的功能，同时也可将所有配置输出 JSON Schema，我们称为**出码**功能

2.3.5 扩展功能与进阶（这部分内容）

历史记录和版本、模版、分享、主题

进阶：协同编辑、定时任务、微前端（集成到其他系统的能力）、混合开发

- 混合开发：组件（ts、flow）和 json 混合开发

- 逻辑的配置：使用流程图（flow），最后生成业务逻辑
- 协同：crdt算法，yjs、ot 等方案

2.4 技术选型与方案选型

- 包管理：pnpm
- 工程化相关
 - vite
 - lint-staged
 - cspell
 - commitizen
 - cz-git
 - husky
 - zx
 - tsno
- lint规范：
 - commitlint;
 - stylelint;
 - prettier;
 - eslint;
 - editorconfig;
- Vue CLI、Vue3
- Pinia
- Vue-Router
- 拖拽库：基于原生的 smooth-dnd 封装用于 Vue3 的拖拽组件
- 编辑器：tiptap for vue3, <https://tiptap.dev/installation/vue3>,
- UI 库：~~Arco Design~~, <https://arco.design/vue/docs/start>
- 表单校验：vee-validate, <https://vee-validate.logaretm.com/v4/>
- 流程编排：@vue-flow/core, <https://vueflow.dev/>
- 图表：echarts

2.5 规范约定

如果你作为前端 Leader 第一需要考虑的就是开发实现成本，但是往往每个成员开发的方式各异，因此我们在项目早期就一定需要做足规范方面的约束。

通常规范约束包含：

- 代码相关：js/ts/style 编码规范
- 命名与单词拼写规范
- git flow 提交规范等

基于此，你需要设计完整的规范约定，并在编码和代码提交时做检测，检测不通过不允许代码提交。

代码相关检查我们使用 eslint 约束 ts 编码，stylelint 约束样式相关。

eslint 项目脚手架已经集成我们不做过多介绍，stylelint 的使用比较简单：

3. 前端所需要掌握的完整技术框架



印客学院2023匠心之作
内容&服务全面升级

Web前端 大厂工程师训练营

 印客2023大厂前端工程师课程大纲.pdf

4. 插件化机制在物料管理中的应用

物料的加载我们选用插件化机制实现，插件化机制在前端应用或者框架中的应用非常广泛。不论是业务开发还是框架开发，我们都能见到插件化设计。比如 webpack 基于 tappable 实现的插件化，babel plugin 的插件化体系等。

插件化机制允许我们为开发者提供更多拓展性

4.1 插件机制引入

我们先通过一个小例子来看看插件化的简单实现：

```
1  class Calculator {
2    construct(initial) {
3      this.num = initial;
4    }
5    add(num) {
6      this.num = this.num + num;
7      return this;
8    }
9    subtract(num) {
10     this.num = this.num - num;
11     return this;
12   }
13   result() {
14     return this.num;
15   }
16 }
17
18 const myCalculator = new Calculator(5);
19 myCalculator.add(5).subtract(2).result(); // 8
```

以上这个例子，我们定义了一个计算器，计算器有加减功能，但是如果我们希望增加乘除功能呢？没办法只能在类中修改代码，这是非常不灵活的，那我们可以怎么做？



做最重要的事！

计算器底座是最重要的事

计算功能作为插件，只需要注册到插件系统即可

插件化机制最重要的事是什么？没错就是插件底座，我们要开发计算器，我们只关心计算器整个底座，至于计算器有什么计算功能，交给外部，外部灵活拓展。顺着这个思路，那么计算器的各种计算方法就是插件，我们需要某个计算功能的时候只需要往插件系统中注册对应插件即可。

```

1  // 程序主体, 定义程序核心逻辑是增加计算器运算能力
2  class Calculator {
3      plugins = [];
4      constructor(initial) {
5          this.num = initial;
6      }
7
8      use(plugin) {
9          this.plugins.push(plugin);
10         this[plugin.name] = plugin.calculate.bind(this);
11     }
12
13     result() {
14         return this.num;
15     }
16 }
17 // 插件声明
18 interface Plugin {
19     name: string;
20     calculate(num: number) => this;
21 }
22 // 插件实现
23 class AddPlugin implements Plugin {
24     name: 'add',
25     calculate(num) {
26         this.num = this.num + num;
27         return this;
28     }
29 }
30
31 class SubtractPlugin implements Plugin {
32     name: 'subtract',
33     calculate(num) {
34         this.num = this.num - num;
35         return this;
36     }
37 }
38
39 const myCalculator = new Calculator(5);
40 // 插件安装
41 myCalculator.use(new AddPlugin());
42 myCalculator.use(new SubtractPlugin());
43
44 myCalculator.add(5).subtract(2).result(); // 8

```

经过这个改造, 未来如果要实现乘法, 我们只需要新增一个插件实现即可, 无需修改程序主体:


```

1 class MultiplicatiPlugin implements Plugin {
2   name: 'multiplicati',
3   calculate(num) {
4     this.num = this.num * num;
5     return this;
6   }
7 }

```

再比如我们要增加一个 help 来打印支持的计算也可以快速实现：

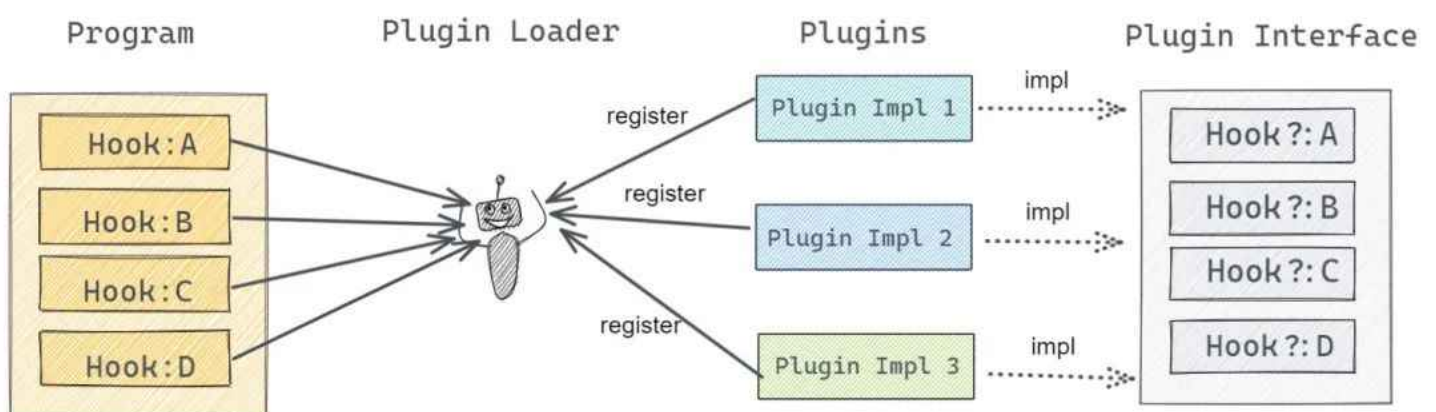
```

1 class Calculator {
2   ...
3   help() {
4     return `support ${this.plugins.map(plugin => plugin.name).join(',')}`;
5   }
6 }

```

通过上面的例子，从插件的角度可以分成几个部分：

- **程序主体（Program）**，我们通常也称之为插件底座，即上例中的 Calculator；
- **插件接口声明（Plugin Interface）**，即上例中的 Plugin；
- **插件实现（Plugin Implementation）**，即 AddPlugin, SubtractPlugin, MultiplicatiPlugin；



插件化机制用途非常广泛：

1. 表单校验，校验规则就可以是插件化的，**rules: [{required: true}, {max: 20}, {min: 10}]**
2. webpack、vue、pinia 这些框架，他的能力拓展都是通过插件化实现的

4.2 物料插件化

我们上一节提到过，物料可以分为基础数据和资源引用，也就是：

```
1 const material = {
2   type: '物料类型',
3   material: '物料资源'
4 }
```

我们的物料资源引用可以来自本地文件，也可以来自在线物料市场，我们暂以本地物料资源为例。

```
1 import type { BlockType } from '@types/block'
2 import QuoteBlock from '@blocks/basic/QuoteBlock.vue'
3 import HeroTitleBlock from '@blocks/basic/HeroTitleBlock.vue'
4 import ViewBlock from '@blocks/basic/ViewBlock.vue'
5 import ChartBlock from '@blocks/basic/ChartBlock.vue'
6 import ImageBlock from '@blocks/basic/ImageBlock.vue'
7 import ButtonBlock from '@blocks/external/ButtonBlock.vue'
8 import FormBlock from '@blocks/external/FormBlock.vue'
9 import NotesBlock from '@blocks/external/NotesBlock.vue'
10 import type { App } from 'vue'
11
12 const baseBlocks = [
13   {
14     type: 'quote',
15     material: QuoteBlock
16   },
17   {
18     type: 'heroTitle',
19     material: HeroTitleBlock
20   },
21   {
22     type: 'view',
23     material: ViewBlock
24   },
25   {
26     type: 'chart',
27     material: ChartBlock
28   },
29   {
30     type: 'image',
31     material: ImageBlock
32   }
33 ]
34 class BlockSuite {
35   private blocks = baseBlocks
```

```

36   constructor() {}
37   getBlocksMap() {
38       return Object.fromEntries(this.blocks.map((block) => [block.type, block]))
39   }
40   getBlocks() {
41       return this.blocks
42   }
43   addBlock(block: any) {
44       this.blocks.push(block)
45   }
46   hasBlock(type: BlockType) {
47       return !!this.getBlocksMap()[type]
48   }
49 }
50
51 const blockSuite = new BlockSuite()
52
53 console.log(
54     '🚀 ~ file: BlockRenderer.vue:55 ~ blockSuite.hasBlock(button):',
55     blockSuite.hasBlock('button')
56 )
57
58 blockSuite.addBlock({
59     type: 'button',
60     material: ButtonBlock
61 })
62 blockSuite.addBlock({
63     type: 'form',
64     material: FormBlock
65 })
66 blockSuite.addBlock({
67     type: 'notes',
68     material: NotesBlock
69 })
70 console.log(
71     '🚀 ~ file: BlockRenderer.vue:68 ~ blockSuite.hasBlock(button):',
72     blockSuite.hasBlock('button')
73 )
74 const blocksMap = blockSuite.getBlocksMap()
75
76 export const blocksMapSymbol = Symbol('blocksMap')
77
78 export const setup = (app: App<Element>) => {
79     const ins = {
80         install(app: App<Element>) {
81             app.provide(blocksMapSymbol, blocksMap)
82             app.config.globalProperties.$blocksMap = blocksMap

```

```

83     }
84   }
85
86   app.use(ins)
87 }
88
89 // Extensions of Vue types to be appended manually
90 // https://github.com/microsoft/rushstack/issues/2090
91 // https://github.com/microsoft/rushstack/issues/1709
92
93 // TODO: figure out why it cannot be 'vue'
94 // @ts-ignore: works on Vue 3, fails in Vue 2
95 declare module '@vue/runtime-core' {
96   export interface ComponentCustomProperties {
97     /**
98      * Access to the application's blocksMap
99      */
100     $blocksMap: string
101   }
102 }

```

4.3 编排

有了物料，我们就可以在编辑器中使用对应物料进行业务页面组装，因此我们需要封装一个物料编排编辑器，对于众多物料，我们首先就会想到需要使用**策略模式**实现不同物料的初始渲染。

```

1 <component :is="xxx" />

```

```

1 <script setup lang="ts">
2 import { toRaw } from 'vue'
3 import { Drag, Delete } from '@icon-park/vue-next'
4 import { useEnvStore } from '@stores/debug'
5 import { useAppEditorStore } from '@stores/appEditor'
6 import { smoothDnD, type DropResult, dropHandlers } from 'smooth-dnd'
7 import { arrayMove } from '@utils/array'
8
9 import { SmoothDndContainer } from '@components/SmoothDnd/SmoothDndContainer'
10 import { SmoothDndDraggable } from '@components/SmoothDnd/SmoothDndDraggable'
11 import { storeToRefs } from 'pinia'
12
13 smoothDnD.dropHandler = dropHandlers.reactDropHandler().handler
14
15 // const props = defineProps<{

```

```

16 // type: BlockType
17 // }>()
18
19 const envStore = useEnvStore()
20 const appEditorStore = useAppEditorStore()
21
22 const { currentBlockId, blocks } = storeToRefs(appEditorStore)
23 const { selectBlock, updateBlocks } = appEditorStore
24
25 const applyDrag = <T extends any[]>(arr: T, dragResult: DropResult) => {
26   const { removedIndex, addedIndex, payload } = dragResult
27
28   const result = [...arr]
29
30   // 没做操作
31   if (addedIndex === null) return result
32
33   // 添加
34   if (addedIndex !== null && removedIndex === null) {
35     result.splice(addedIndex, 0, {
36       id: `${Math.random()}`,
37       ...payload
38     })
39   }
40
41   // 移动
42   if (addedIndex !== null && removedIndex !== null) {
43     return arrayMove(result, removedIndex, addedIndex)
44   }
45
46   return result
47 }
48 </script>
49
50 <template>
51   <smooth-dnd-container
52     drag-handle-selector=".handle"
53     group-name="blocks"
54     orientation="vertical"
55     tag="div"
56     @drop="updateBlocks(applyDrag(toRaw(blocks), $event))"
57   >
58     <smooth-dnd-draggable v-for="(block, i) in blocks" :key="block.id">
59       <div class="block-wrapper" @click.stop="selectBlock(block.id)">
60         <!-- @vue-ignore -->
61         <component :is="$blocksMap[block.type].material" class="block" />
62         <div

```

```

63         :class="[
64             'block-wrapper-indicator',
65             { shown: envStore.debug, selected: currentBlockId === block.id }
66         ]"
67     >
68         <div class="block-toolbar" v-if="currentBlockId === block.id">
69             <div class="block-toolbar-item handle">
70                 <drag />
71             </div>
72             <div class="block-toolbar-item" @click="blocks.splice(i, 1)">
73                 <delete />
74             </div>
75         </div>
76     </div>
77 </div>
78 </smooth-dnd-draggable>
79 </smooth-dnd-container>
80 <!-- <div ref="containerRef">
81     <div
82         class="block-wrapper"
83         v-for="(d, i) in b"
84         :key="d.id"
85         @click.stop="appEditorStore.selectBlock(d.id)"
86     >
87         <component :is="d.material" class="block" />
88         <div
89             :class="[
90                 'block-wrapper-indicator',
91                 { shown: envStore.debug, selected: appEditorStore.currentBlockId === d.id }
92             ]"
93         >
94             <div class="block-toolbar" v-if="appEditorStore.currentBlockId === d.id">
95                 <div class="block-toolbar-item handle">
96                     <drag />
97                 </div>
98                 <div class="block-toolbar-item" @click="b.splice(i, 1)">
99                     <delete />
100                 </div>
101             </div>
102         </div>
103     </div>
104 </div> -->
105 </template>
106
107 <style scoped>
108 .block-wrapper {
109     position: relative;

```

```
110 display: flex;
111 width: 100%;
112 margin-top: 16px;
113 padding: 6px 4px;
114 border-radius: 8px;
115 background-color: var(--color-white);
116 transition: box-shadow 0.2s ease-in-out;
117 }
118
119 .block {
120 position: relative;
121 z-index: 1;
122 }
123
124 .block-wrapper-indicator {
125 content: '';
126 position: absolute;
127 width: 100%;
128 height: 100%;
129 left: 0;
130 top: 0;
131 border-radius: 8px;
132 pointer-events: none;
133 user-select: none;
134 }
135
136 .block-wrapper-indicator.shown {
137 border: 1px dashed var(--color-gray-300);
138 }
139
140 .block-wrapper-indicator.selected {
141 border: 1px solid var(--color-primary);
142 }
143
144 .block-toolbar {
145 display: flex;
146 align-items: center;
147 position: absolute;
148 left: 0;
149 top: -36px;
150 z-index: 1;
151 padding: 4px 8px;
152 gap: 4px;
153 background-color: var(--color-black);
154 border-radius: 6px;
155 pointer-events: visible;
156 }
```

```
157
158 .block-toolbar-item {
159     display: flex;
160     justify-content: center;
161     align-items: center;
162     width: 22px;
163     height: 22px;
164     border-radius: 4px;
165     color: var(--color-white);
166     cursor: pointer;
167 }
168
169 .block-toolbar-item:nth-of-type(1) {
170     cursor: grab;
171 }
172
173 .block-toolbar-item:hover {
174     background-color: var(--color-gray-800);
175     transition: all 0.2s ease-in-out;
176 }
177
178 .block-wrapper.debug:hover .block-wrapper-senior {
179     border-style: solid;
180     transition: box-shadow 0.2s ease-in-out;
181 }
182 </style>
183
184 <style lang="css">
185 .smooth-dnd-draggable-wrapper {
186     overflow: visible !important;
187 }
188 </style>
189
```

5. 渲染引擎设计与实现

5.1 设计物料 JSON Schema

我们需要针对多样物料设计上层趋于一致的结构，这样不论是在物料编排还是出料，都可以保证数据的高度规范，有利于数据类型推导与渲染器的功能编写。

通常物料我们需要关注一下重点属性，或者说通常这个 json 我们这样设计，基本上可以满足常规需求，当然如果有更丰富的功能也可以在此基础上添加定义即可。


```

1  const materialSchema = {
2    id: '1',
3    type: 'quote',
4    label: '引述',
5    props: {
6      title: '引述文本 quote',
7      // ... 其他类型物料可以在这里有更多属性定义
8    },
9    events: {
10     onClick(ev, ctx, item) {
11       console.log(ev)
12     }
13   },
14   attrs: {},
15   // children: [] 如果你的编排选择使用树状组织，则有可能需要通过 children 知道后代元素，
16   // 当然你也可以选择平铺数据表示
17 }

```

5.2 定义物料插件

我们前面提到物料及插件化体系，我们新的物料都将抽象为插件。

插件描述物料基本数据特征，配合插件模板实现完整单物料组件能力。

```

1  {
2    id: '1',
3    type: 'quote',
4    label: '引述',
5    props: {
6      content: '引述文本 quote',
7      status: 'success'
8    },
9    events: {
10     triggerFocus() {
11       // ...
12     }
13   }
14 }

```

物料插件基础数据用来描述物料在整个应用中的特征，如上数据结构所描述的，引述文本物料包含基本特性：id、type、label，除此之外，引述文本还包含特有属性，例如：引述文本的内容、引述文本的状态。

5.3 定义物料模板

物料模板用来充分描述物料在应用中的 UI 表现。

```
1 <script setup lang="ts">
2 import { Error, Success, Wallet } from '@icon-park/vue-next'
3
4 import type { QuoteBlockInfo } from '@types/block'
5
6 const STATUS_MAP = {
7   success: {
8     color: '#059669',
9     bgColor: '#E4F2E9',
10    borderColor: '#79DBA2',
11    icon: Success
12  },
13  warning: {
14    color: '#E18F05',
15    bgColor: '#F8ECEC',
16    borderColor: '#79DBA2',
17    icon: Error
18  },
19  error: {
20    color: '#DC2C26',
21    bgColor: '#F8ECEC',
22    borderColor: '#F3A9A5',
23    icon: Wallet
24  }
25 }
26
27 const props = defineProps<{
28   blockInfo: QuoteBlockInfo
29 }>()
30
31 const { color, bgColor, /* borderColor, */ icon } = STATUS_MAP[props.blockInfo.p
32 </script>
33
34 <template>
35   <div
36     class="quote"
37     :style="{ backgroundColor: bgColor, color /* , border: `1px solid ${borderCo
38   >
39     <component :is="icon" />
40     <span class="quote-text">{{ props.blockInfo.props.content }}</span>
```

```

41   </div>
42 </template>
43
44 <style scoped>
45   .quote {
46     display: flex;
47     align-items: center;
48     flex: 1;
49     font-size: var(--font-size-large);
50     padding: 8px 18px;
51     border-radius: 8px;
52
53     /* background-color: ; */
54   }
55
56   .quote-text {
57     margin-left: 6px;
58   }
59 </style>
60

```

5.4 物料渲染器

物料渲染器主要负责解析 JSON Schema，并将其用不同物料模板渲染。

简单来说，我们需要使用策略模式，根据物料类型，使用对应物料模板进行渲染并对该物料进行事件绑定与其他处理。

```

1  <script setup lang="ts">
2  import { Delete, Drag } from '@icon-park/vue-next'
3  import { dropHandlers, smoothDnD } from 'smooth-dnd'
4  import { inject, ref } from 'vue'
5
6  import { useAppEditorStore } from '@stores/appEditor'
7  import { useEnvStore } from '@stores/debug'
8
9  // import { useClickOutside } from '@hooks/useClickOutside'
10
11  defineProps<{
12    block: BlockInfo
13    i: number
14  }>()
15
16  import { storeToRefs } from 'pinia'
17

```

```

18 import type { BlockInfo } from '@/types/block'
19
20 smoothDnD.dropHandler = dropHandlers.reactDropHandler().handler
21
22 // const props = defineProps<{
23 //   type: BlockType
24 // }>()
25 const blockWrapperRef = ref<HTMLElement | null>(null)
26 const envStore = useEnvStore()
27 const appEditorStore = useAppEditorStore()
28
29 const { currentBlockId, blocks } = storeToRefs(appEditorStore)
30 const { selectBlock } = appEditorStore
31
32 // 需要保证 blocksMap 在 BlockRenderer 之前被注入，并且我们暂时使用的 Symbol 作为 key
33 // console.log(inject('blocksMap'))
34 // useClickOutside(blockWrapperRef)
35 const editable = inject('editable', true)
36 </script>
37
38 <template>
39   <div class="block-wrapper" ref="blockWrapperRef" @click.stop="selectBlock(blockInfo)">
40     <!-- @vue-ignore -->
41     <component :is="$blocksMap[block.type].material" class="block" :blockInfo="blockInfo">
42       <div
43         v-if="editable"
44         :class="[
45           'block-wrapper-indicator',
46           { shown: envStore.debug, selected: currentBlockId === block.id }
47         ]"
48       >
49         <div class="block-toolbar" v-if="currentBlockId === block.id">
50           <div class="block-toolbar-item handle">
51             <drag />
52           </div>
53           <div class="block-toolbar-item" @click="blocks.splice(i, 1)">
54             <delete />
55           </div>
56         </div>
57       </div>
58     </div>
59 </template>
60
61 <style scoped>
62 .block-wrapper {
63   position: relative;
64   display: flex;

```

```
65   width: 100%;
66   margin-top: 16px;
67   padding: 6px 4px;
68   border-radius: 8px;
69   background-color: var(--color-white);
70   transition: box-shadow 0.2s ease-in-out;
71 }
72
73 .block {
74   position: relative;
75   z-index: 1;
76 }
77
78 .block-wrapper-indicator {
79   content: '';
80   position: absolute;
81   width: 100%;
82   height: 100%;
83   left: 0;
84   top: 0;
85   border-radius: 8px;
86   pointer-events: none;
87   user-select: none;
88 }
89
90 .block-wrapper-indicator.shown {
91   border: 1px dashed var(--color-gray-300);
92 }
93
94 .block-wrapper-indicator.selected {
95   border: 1px solid var(--color-primary);
96 }
97
98 .block-toolbar {
99   display: flex;
100  align-items: center;
101  position: absolute;
102  left: 0;
103  top: -36px;
104  z-index: 1;
105  padding: 4px 8px;
106  gap: 4px;
107  background-color: var(--color-black);
108  border-radius: 6px;
109  pointer-events: visible;
110 }
111
```

```
112 .block-toolbar-item {
113     display: flex;
114     justify-content: center;
115     align-items: center;
116     width: 22px;
117     height: 22px;
118     border-radius: 4px;
119     color: var(--color-white);
120     cursor: pointer;
121 }
122
123 .block-toolbar-item:nth-of-type(1) {
124     cursor: grab;
125 }
126
127 .block-toolbar-item:hover {
128     background-color: var(--color-gray-800);
129     transition: all 0.2s ease-in-out;
130 }
131
132 .block-wrapper.debug:hover .block-wrapper-senior {
133     border-style: solid;
134     transition: box-shadow 0.2s ease-in-out;
135 }
136 </style>
137
138 <style lang="css">
139 .smooth-dnd-draggable-wrapper {
140     overflow: visible !important;
141 }
142 </style>
143
```